

Docker Complete Beginner's Guide

Everything you need to know about Docker for this project

🤔 What is Docker? (Explained Like You're Five)

The Shipping Container Analogy

Imagine you're shipping furniture across the country:

Without Docker (Traditional Shipping):

- Pack each piece differently
- Different truck for each item
- Hope nothing breaks
- Receiving end has to figure out assembly
- "It worked fine at the factory!" 🙄

With Docker (Containerized Shipping):

- Everything packed in standard containers
- Any truck can carry any container
- Protected and organized
- Receiving end just opens container - everything's ready
- Works the same everywhere ✅

In Software Terms

Your laptop:

- Python 3.11
- pandas 2.1.0
- Your code

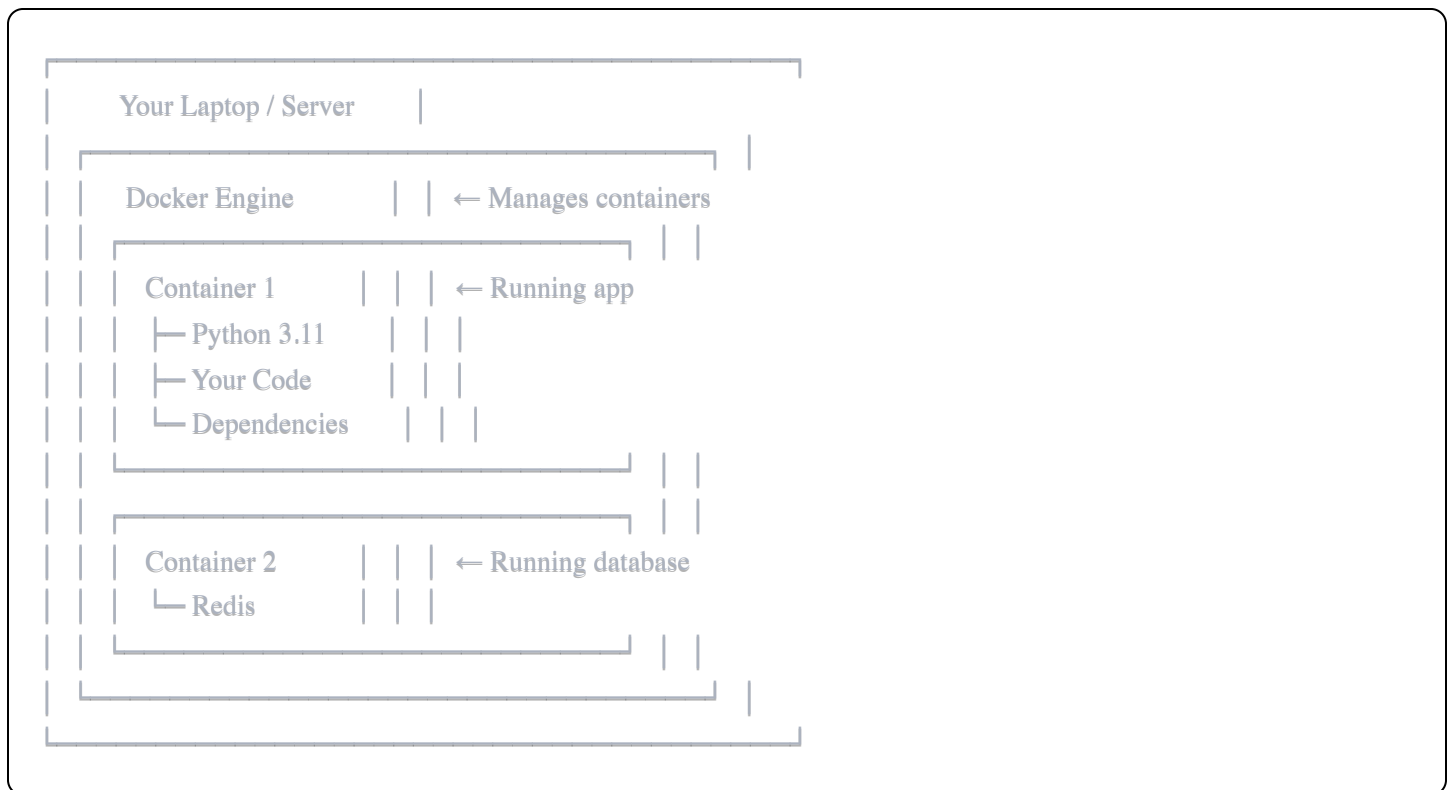
Production server:

- Python 3.9 (oops, wrong version!)
- pandas 1.5.0 (oops, old version!)
- Your code (doesn't work! 😭)

With Docker:

- Container includes: Python 3.11 + pandas 2.1.0 + your code
- Works on your laptop, works on server, works everywhere! 🎉

📦 Docker Mental Model



Key Concepts:

1. **Image** = Recipe/Blueprint (read-only)
 - Like: A cake recipe
 - Example: `python:3.11-slim` image
2. **Container** = Running instance (writable)
 - Like: An actual baked cake
 - Example: Your running app

3. **Dockerfile** = Instructions to build image

- Like: Recipe card
- Example: The Dockerfile in this project

4. **Registry** = Image storage

- Like: Cookbook library
- Example: Docker Hub, AWS ECR



Installation

Mac

```
bash

# Download Docker Desktop from:
# https://www.docker.com/products/docker-desktop

# Or using Homebrew:
brew install --cask docker

# Start Docker Desktop app
# Look for whale icon in menu bar
```

Windows

```
bash

# Download Docker Desktop from:
# https://www.docker.com/products/docker-desktop

# IMPORTANT: Enable WSL 2 (Windows Subsystem for Linux)
# Docker Desktop will guide you through this

# Start Docker Desktop
# Look for whale icon in system tray
```

Linux (Ubuntu)

```
bash
```

Update package index

```
sudo apt-get update
```

Install prerequisites

```
sudo apt-get install -y \  
    apt-transport-https \  
    ca-certificates \  
    curl \  
    gnupg \  
    lsb-release
```

Add Docker's official GPG key

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring
```

Set up repository

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker
```

```
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Install Docker Engine

```
sudo apt-get update  
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
```

Add your user to docker group (avoid needing sudo)

```
sudo usermod -aG docker $USER
```

Log out and back in for group change to take effect

Verify Installation

```
bash
```

Check Docker version

```
docker --version
```

Output: Docker version 24.0.6, build ed223bc

Test installation

```
docker run hello-world
```

Should download and run a test container

🎓 Docker Commands Tutorial

Understanding Command Structure

bash

`docker <COMMAND> <OPTIONS> <ARGUMENTS>`

↑

↑

↑

what to do how to do parameter

Basic Commands

1. Build an Image

bash

Basic build

`docker build -t my-app .`

#

↑

↑

↑

| | └─ Where to find Dockerfile (.= current directory)

| └────────── Name/tag for the image

└────────── Build command

Real example for this project:

`docker build -t portfolio-analytics .`

What happens:

1. Docker reads Dockerfile

2. Executes each instruction (FROM, RUN, COPY, etc.)

3. Creates layers (cached for speed)

4. Tags final image as "portfolio-analytics"

5. Image is now on your machine

Build with custom Dockerfile location:

`docker build -t my-app -f /path/to/Dockerfile .`

Build with build arguments:

`docker build --build-arg PYTHON_VERSION=3.12 -t my-app .`

2. List Images

bash

Show all images on your machine

`docker images`

Output example:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
portfolio-analytics	latest	abc123def456	5 minutes ago	420MB
python	3.11	789ghi012jkl	2 weeks ago	150MB

Clean up unused images:

`docker image prune`

3. Run a Container

`bash`

Basic run (container exits after command finishes)

```
docker run portfolio-analytics
```

What happens:

1. Docker creates a container from the image

2. Runs the CMD from Dockerfile (batch_processor.py)

3. Container exits when script finishes

Run with environment variables:

```
docker run \
  -e MONGO_URI="mongodb://..." \
  -e REDIS_URL="redis://..." \
  portfolio-analytics
```

Run in background (detached mode):

```
docker run -d portfolio-analytics
```

```
#      ↑
#      └─ detached (runs in background)
```

Run with port mapping (for dashboard):

```
docker run -p 8050:8050 portfolio-analytics
```

```
#      ↑ ↑ ↑
#      | | └─ Container port
#      | └─ Host port (your laptop)
#      └─ Port mapping flag
```

Run interactively (get a shell inside container):

```
docker run -it portfolio-analytics /bin/bash
```

```
#      ↑↑
#      | |
#      | └─ Allocate terminal
#      └─ Interactive mode
```

Now you're INSIDE the container! Try:

```
pwd      # See current directory
```

```
ls        # List files
```

```
python --version
```

```
exit      # Leave container
```

4. List Running Containers

```
bash
```

Show running containers

`docker ps`

Output example:

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
abc123def456	portfolio-analytics	Up 2 minutes		cool_name

Show ALL containers (including stopped)

`docker ps -a`

Show only container IDs (useful for scripting)

`docker ps -q`

5. Container Lifecycle

bash

Stop a running container

`docker stop <container-id>`

or

`docker stop abc123def456`

Start a stopped container

`docker start <container-id>`

Restart a container

`docker restart <container-id>`

Remove a container (must be stopped first)

`docker rm <container-id>`

Force remove (even if running)

`docker rm -f <container-id>`

Remove all stopped containers

`docker container prune`

6. View Logs

bash

See output from a container

`docker logs <container-id>`

Follow logs in real-time (like tail -f)

`docker logs -f <container-id>`

Last 50 lines

`docker logs --tail 50 <container-id>`

Logs with timestamps

`docker logs -t <container-id>`

7. Execute Commands in Running Container

bash

Run a command in a running container

`docker exec <container-id> <command>`

Example: Check Python version in running container

`docker exec abc123def456 python --version`

Get a shell in running container

`docker exec -it <container-id> /bin/bash`

This is like SSH-ing into the container

8. Copy Files To/From Container

bash

Copy FROM container to host

`docker cp <container-id>:/app/output.csv ./local-output.csv`

Copy TO container from host

`docker cp ./local-file.txt <container-id>:/app/file.txt`

9. Inspect Container/Image

bash

Get detailed info about a container

`docker inspect <container-id>`

Get detailed info about an image

`docker inspect portfolio-analytics`

Shows: environment vars, ports, volumes, etc.

Dockerfile Deep Dive

Let's build a Dockerfile from scratch and understand each line:

```
dockerfile
```

```
# =====
# Line 1: Base Image
# =====

FROM python:3.11-slim

# What this does:
# - Downloads python:3.11-slim image from Docker Hub (if not already local)
# - This image contains:
#   ✓ Ubuntu Linux (minimal)
#   ✓ Python 3.11 installed
#   ✓ pip installed
# - This is your starting point

# =====
# Lines 2-3: Environment Variables
# =====

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# What this does:
# PYTHONDONTWRITEBYTECODE=1
# → Prevents Python from creating .pyc files (compiled bytecode)
# → Why? Saves space, not needed in containers
#
# PYTHONUNBUFFERED=1
# → Makes Python output appear immediately
# → Why? See logs in real-time, not buffered

# =====
# Line 4: Working Directory
# =====

WORKDIR /app

# What this does:
# - Creates /app directory in container
# - Sets it as working directory (like cd /app)
# - All subsequent commands run from /app

# =====
# Lines 5-6: Install Dependencies (Layer Caching!)
```

```
# =====
```

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

What this does:

COPY requirements.txt .

→ Copies ONLY requirements.txt from your computer to /app

→ Why only this file? LAYER CACHING!

#

Layer caching explained:

- Each instruction creates a "layer"

- Docker caches layers

- If file doesn't change, Docker reuses cached layer

#

Workflow:

1st build: Installs all packages (slow, ~60 seconds)

2nd build (code changed, requirements.txt unchanged):

→ Reuses cached package layer (fast, ~5 seconds!)

#

If we did "COPY . ." first, ANY code change would invalidate

the cache and reinstall ALL packages!

```
# =====
```

Line 7: Copy Application Code

```
# =====
```

COPY . .

What this does:

- Copies everything from current directory to /app

- This layer invalidates when ANY file changes

- That's fine because code is copied AFTER packages are installed

```
# =====
```

Line 8: Default Command

```
# =====
```

CMD ["python", "-u", "batch_processor.py"]

What this does:

- Defines what runs when you start a container

- Array format: ["command", "arg1", "arg2"]

```
# -u flag: Unbuffered Python output
# - Can be overridden: docker run image-name python other_script.py
```

Build Process Visualization

Docker Build Process:

Instruction	Layer	Cached?
FROM python:3.11-slim	Base image	✓ (pulled once)
ENV PYTHONUNBUFFERED=1	Layer 1	✓ (never changes)
WORKDIR /app	Layer 2	✓ (never changes)
COPY requirements.txt .	Layer 3	✓ (if file unchanged)
RUN pip install ...	Layer 4	✓ (depends on Layer 3)
COPY . .	Layer 5	✗ (changes frequently)
CMD [...]	Layer 6	✓ (metadata only)

Result: Fast rebuilds! Only Layer 5 rebuilds when code changes.

💡 Common Docker Workflows

Workflow 1: Local Development

```
bash

# 1. Make code changes
vim batch_processor.py

# 2. Rebuild image
docker build -t portfolio-analytics .

# 3. Run container with .env file
docker run --env-file .env portfolio-analytics

# 4. Check logs
docker logs <container-id>
```

Workflow 2: Debugging

```
bash

# 1. Run container interactively
docker run -it --env-file .env portfolio-analytics /bin/bash

# 2. Now you're inside! Try things:
python batch_processor.py    # Run script manually
python -c "import pymongo"   # Test imports
cat /app/requirements.txt    # Check files
env | grep MONGO             # Check env vars

# 3. Exit when done
exit
```

Workflow 3: Testing on Different Python Versions

```
dockerfile

# Dockerfile.py312
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "batch_processor.py"]
```

```
bash

# Build for Python 3.12
docker build -f Dockerfile.py312 -t portfolio-analytics:py312 .

# Test
docker run --env-file .env portfolio-analytics:py312
```

Workflow 4: Production Deployment

```
bash
```

1. Build production image

```
docker build -t my-registry/portfolio-analytics:v1.0 .
```

2. Push to registry

```
docker push my-registry/portfolio-analytics:v1.0
```

3. On production server:

```
docker pull my-registry/portfolio-analytics:v1.0
```

```
docker run -d --restart=unless-stopped \
```

```
-e MONGO_URI="..." \
```

```
-e REDIS_URL="..." \
```

```
my-registry/portfolio-analytics:v1.0
```

Troubleshooting

Problem: "Cannot connect to Docker daemon"

```
bash
```

Check if Docker is running

```
docker info
```

Mac: Open Docker Desktop app

Linux: Start Docker service

```
sudo systemctl start docker
```

Problem: "Port already in use"

```
bash
```

Find what's using port 8050

```
lsof -i :8050
```

Use a different port

```
docker run -p 8051:8050 portfolio-analytics
```

Problem: "No space left on device"

```
bash
```

Clean up unused images

`docker image prune -a`

Clean up unused containers

`docker container prune`

Clean up everything (careful!)

`docker system prune -a`

Problem: "Permission denied"

bash

Linux: Add yourself to docker group

`sudo usermod -aG docker $USER`

Log out and back in, then test:

`docker run hello-world`

Problem: Build is slow

bash

Create .dockerignore to exclude files

`cat > .dockerignore << EOF`

`__pycache__/`

`*.pyc`

`.git/`

`.env`

`venv/`

`*.log`

`EOF`

Now rebuild

`docker build -t portfolio-analytics .`

Next Steps

1. Practice Commands

```
bash
```

```
# Try these exercises:
```

```
# Exercise 1: Build and run
```

```
docker build -t test-app .
```

```
docker run test-app
```

```
# Exercise 2: Interactive exploration
```

```
docker run -it test-app /bin/bash
```

```
ls
```

```
python --version
```

```
exit
```

```
# Exercise 3: View logs
```

```
docker run -d test-app
```

```
docker logs <container-id>
```

```
# Exercise 4: Clean up
```

```
docker ps -a
```

```
docker rm <container-id>
```

```
docker images
```

```
docker rmi test-app
```

2. Read the Dockerfile

Go through the Dockerfile line by line and understand what each instruction does.

3. Modify the Dockerfile

Try adding:

```
dockerfile
```

```
# Install additional package
```

```
RUN pip install matplotlib
```

```
# Create a directory
```

```
RUN mkdir /app/logs
```

```
# Set a timezone
```

```
ENV TZ=America/New_York
```

4. Multi-Container Setup

Learn Docker Compose to run multiple services (app + Redis):

```
yml
```

```
# docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
  app:
```

```
    build: .
```

```
    environment:
```

```
      - REDIS_URL=redis://redis:6379
```

```
  redis:
```

```
    image: redis:alpine
```

```
bash
```

```
docker-compose up
```

Docker Best Practices

1. **Use .dockerignore** - Exclude unnecessary files
2. **Leverage layer caching** - Copy requirements.txt before code
3. **Use specific tags** - `python:3.11-slim` not `python:latest`
4. **Run as non-root** - Add USER instruction
5. **Keep images small** - Use slim/alpine variants
6. **One process per container** - Don't run multiple services

7. **Use environment variables** - Not hardcoded values
 8. **Add HEALTHCHECK** - Monitor container health
 9. **Multi-stage builds** - Smaller production images
 10. **Security scan** - Use `docker scan` to check vulnerabilities
-

Additional Resources

- **Official Docker Docs:** <https://docs.docker.com/>
 - **Docker Cheat Sheet:** <https://dockerlabs.collabnix.com/docker/cheatsheet/>
 - **Play with Docker:** <https://labs.play-with-docker.com/>
 - **Docker Tutorial:** <https://www.docker.com/101-tutorial/>
 - **Dockerfile Best Practices:** https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
-

You now know enough Docker to containerize any project! 🎉

Remember: Docker is a skill that pays dividends - every tech company uses it!